



SVKM's NMIMS, Indore

School of Technology & Management in Engineering, Indore

Image and Video Processing

Course Project Report

FRIDAY: A Lightweight Vision-Based Personal Desktop Assistant Using Real-Time Face, Gesture, and Image Processing

Student Name	Khwaish Verma, Anshul Pathak, Devi Ratan Singh
SAP ID	70022400265, 70022400676, 70022400737
Programme / Semester	B.Tech. (Computer Engineering), Section-A, Semester V
Batch	A1
Faculty Name	Dr. Raj Gaurav Mishra
Institute / Department	SVKM's NMIMS, School of Technology & Management in Engineering, Indore
Academic Year	2026–27

Abstract

Conventional desktop interaction depends on physical input devices such as the keyboard and mouse. This project addresses the problem of building a more natural, hands-free and secure desktop interface using only a webcam. FRIDAY is a vision-based personal desktop assistant that combines face-based authentication, hand-gesture control, a set of live image-processing tools, touchless virtual musical instruments, productivity tools and a Gemini-powered chat assistant in a single PyQt6 application. It can also hand over to Mark LIV (JARVIS), a separate voice-first assistant bundled in the same repository.

Webcam frames are acquired with OpenCV. Authentication uses Haar-cascade face detection, grayscale conversion, resizing and histogram equalisation, followed by LBPH face recognition and verification over eight consecutive frames. Gesture control uses the MediaPipe Hand Landmarker to obtain 21 hand landmarks per hand, from which finger states and a scale-normalised pinch ratio are computed; a rule-based classifier labels eight gestures, a four-frame temporal stabilizer suppresses flicker, and a two-hand "two thumbs up" gesture captures a screenshot with an on-screen notification. A Mini Vision Studio applies grayscale conversion, Gaussian blur, Canny edge detection, thresholding, sharpening, HSV conversion and negative transformation to live video, while an Air Piano, Air Guitar and Air Tabla turn the index fingertip into a musical controller.

The outcome is a working real-time system in which a registered user unlocks the dashboard by face, moves the cursor with the index finger, clicks with a pinch, opens the chat assistant with two fingers and captures screenshots with two thumbs up. Screenshots of the working application and the measurements that could be recorded are reported in the Results section.

1. Introduction

1.1 Background of the problem

Human-computer interaction has long been dominated by keyboards, mice and touchscreens. Inexpensive webcams and efficient computer-vision libraries now make it practical for a computer to "see" its user, so that a face can serve as a credential and a hand can serve as a pointing device. A webcam produces a continuous sequence of frames, so such a system is a video-processing problem rather than a single-image problem.

1.2 Relevance of the problem

Vision-based interfaces are relevant to hands-free computing, accessibility, touchless control in settings where touching shared devices is undesirable, and lightweight biometric login that needs no extra hardware. Running the vision pipeline locally on a laptop CPU with a standard webcam keeps the system accessible to ordinary users.

1.3 Role of image and video processing

Every interaction in FRIDAY begins with image and video processing: frame acquisition, colour-space conversion, histogram equalisation, face detection, landmark-based feature extraction and classification. Temporal processing across frames (multi-frame authentication and gesture stabilization) turns noisy per-frame results into reliable decisions. The same landmark stream also drives the virtual instruments, where fingertip position is mapped to piano keys, guitar strings and tabla zones.

1.4 Motivation

The project was chosen to apply the concepts of the course (acquisition, filtering, thresholding, edge detection, feature extraction, detection, recognition and video processing) in one interactive application instead of isolated laboratory exercises. Wrapping them in a complete desktop assistant, with a login screen, dashboard and tools, shows how these techniques combine into a usable product.

2. Problem Statement and Objectives

2.1 Problem statement

To design and implement a lightweight, real-time, vision-based desktop assistant that uses webcam video to provide secure face authentication and natural hand-gesture interaction, together with image-processing tools, touchless instruments, productivity utilities and an AI assistant in an integrated environment.

2.2 Objectives

1. **Vision-based authentication:** detect and recognise a registered user from the webcam, and grant access only after consistent multi-frame verification.
2. **Real-time hand tracking:** extract 21 hand landmarks per frame and classify a small, reliable vocabulary of gestures.
3. **Hands-free interaction:** map gestures to desktop actions (cursor movement, click, chat toggle, screenshot) with temporal stabilization to avoid accidental triggers.
4. **Image processing:** implement seven operations (grayscale, blur, edge, threshold, sharpen, HSV, negative) on live video in the Mini Vision Studio.
5. **Touchless instruments:** play an Air Piano, Air Guitar and Air Tabla using fingertip tracking and synthesised audio.
6. **Assistant and productivity tools:** provide a Gemini chat assistant with voice input and spoken replies, a media controller, calendar, notes, utility tools and a system monitor.
7. **Integration:** combine the modules into a PyQt6 dashboard with managed camera ownership, and bridge to the separate Mark LIV (JARVIS) assistant.

3. Scope of the Project

3.1 What the project covers

- **Face authentication:** face detection, recognition and multi-frame verification; handling of no-face and multiple-face cases; local face enrollment and training scripts.
- **Hand gestures:** hand detection, 21-point landmarks, finger-state extraction, eight-gesture classification, temporal stabilization and a two-hand screenshot gesture.
- **Air mouse:** index-fingertip cursor control with smoothing and dead-zone, and pinch-to-click.
- **Mini Vision Studio and Gesture Lab:** live processing modes, FPS display, image capture, and live display of the recognised gesture.
- **Virtual instruments:** Air Piano, Air Guitar and Air Tabla with pre-synthesised audio.
- **Dashboard modules:** system monitor, media controller, calendar, notes, tools (calculator, unit converter, timer, stopwatch, QR generator) and the FRIDAY chat assistant with voice input and narration.
- **Notification toast:** floating on-screen confirmation after a gesture-triggered screenshot.
- **Mark LIV (JARVIS) launcher:** one-click hand-over to the bundled voice assistant.

3.2 What the project does not cover

- Large-scale face datasets, deep-learning face recognition and production-grade biometric security (LBPH is used for a small number of local users and is not designed to resist spoofing with photos or video).
- Training of new gesture or landmark models: the pretrained MediaPipe model is used as is.
- Dynamic (motion-based) gestures and multi-user gesture control: the pointer and dashboard gestures use only the first detected hand; the second hand is used only for the two-thumbs-up screenshot gesture.
- Development of the Mark LIV assistant itself: it is an open-source project bundled with FRIDAY and only launched from it.

3.3 Type of images / videos used

Live colour (BGR) video from a laptop webcam, plus locally stored grayscale face crops of the registered user. Screenshots captured by the gesture are saved as PNG. No pre-recorded video dataset is used.

3.4 Application domain

Computer vision, image and video processing, human–computer interaction, assistive and touchless computing, creative/musical interfaces, and intelligent desktop systems.

4. Dataset / Input Description

FRIDAY is a real-time system and its inputs are described below.

Item	Description
Source	Laptop webcam (OpenCV VideoCapture, camera index 0); frames are mirrored horizontally.
Dataset name	None (live capture). Face samples are collected by the project's registration script.
Face samples	20 images per user are captured by register_face.py, only when exactly one face is visible. Stored under data/faces/<name>/.
Resolution	Capture is requested at 1280×720 ; face crops are resized to 200×200 for recognition.
Format	Live frames as 8-bit BGR arrays; face samples saved as grayscale images; screenshots saved as PNG in data/screenshots/ (FRIDAY_<date>_<time>.png); Vision Studio captures saved as PNG.
Video duration	Continuous live stream (no fixed duration).
Hand input	No image dataset: the MediaPipe Hand Landmarker (hand_landmarker.task, about 7.8 MB) returns 21 landmarks with normalised x, y, z coordinates for up to two hands.
Other local data	Notes (data/notes.json), calendar events (data/calendar_events.json) and the trained face model, all kept on the user's machine.

5. Methodology / Proposed Approach

The system runs two vision pipelines on frames from a shared camera abstraction. The authentication pipeline runs on the lock screen; after a successful login, the gesture pipeline runs in background so that the interface is never blocked. The module structure is shown in Figure 3.

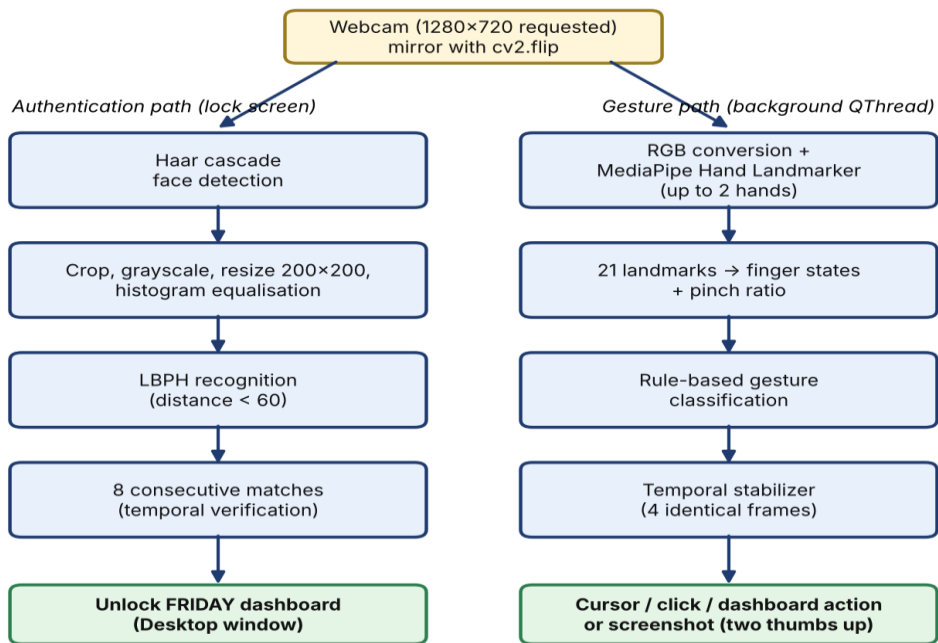


Figure 1: Overall processing pipeline of FRIDAY

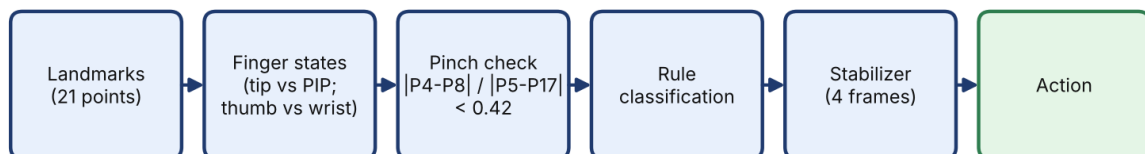
5.1 Authentication algorithm

```
for every frame (timer, 30 ms):
    faces = detect_faces(gray(frame))
    if len(faces) == 0: reset(); status = LOCKED
    elif len(faces) > 1: reset(); status = MULTIPLE FACES
    else:
        face = preprocess(crop)      # gray, 200x200, equalizeHist
        name, distance = LBPH.predict(face)
        if name is Unknown or distance >= 60: reset(); DENIED
        else:
            match_count += 1
            if match_count >= 8: ACCESS GRANTED -> open dashboard
```

5.2 Gesture algorithm

```
for every frame:
    result = HandLandmarker.detect(RGB(frame))      # up to 2 hands
    gestures = [classify(detect_fingers(h), h) for h in hands]
    if both hands are THUMBS_UP for 6 frames:
        emit TWO_THUMBS_UP                          # screenshot
    elif a hand is present:
        raw = gestures[0]
        stable = stabilizer.update(raw)              # 4 identical frames
        execute(raw if raw == POINT else stable, index_tip)
        emit stable gesture when it changes
    else:
        stabilizer.reset(); actions.reset()
```

Gesture classification pipeline



Output classes: POINT, PINCH, TWO_FINGER, THREE_FINGER, FOUR_FINGER, OPEN_PALM, FIST, THUMBS_UP, UNKNOWN

Two hands: both THUMBS_UP held for 6 frames → TWO_THUMBS_UP (screenshot)

Figure 2: Gesture classification pipeline

5.3 System architecture and camera arbitration

main.py creates the FridayApplication, which shows the LockScreen full-screen. When the lock screen emits an authenticated signal, it is closed and the Desktop window opens with the sidebar and workspace pages shown in Figure 3. Because a webcam can normally be opened by only one component at a time, the Desktop pauses the background gesture worker whenever a camera-hungry page (Vision Studio, Musical Instruments) is opened, lets that page own camera 0, and restarts gesture control when the page is closed or the user returns Home. The same hand-over happens before Mark LIV is launched.

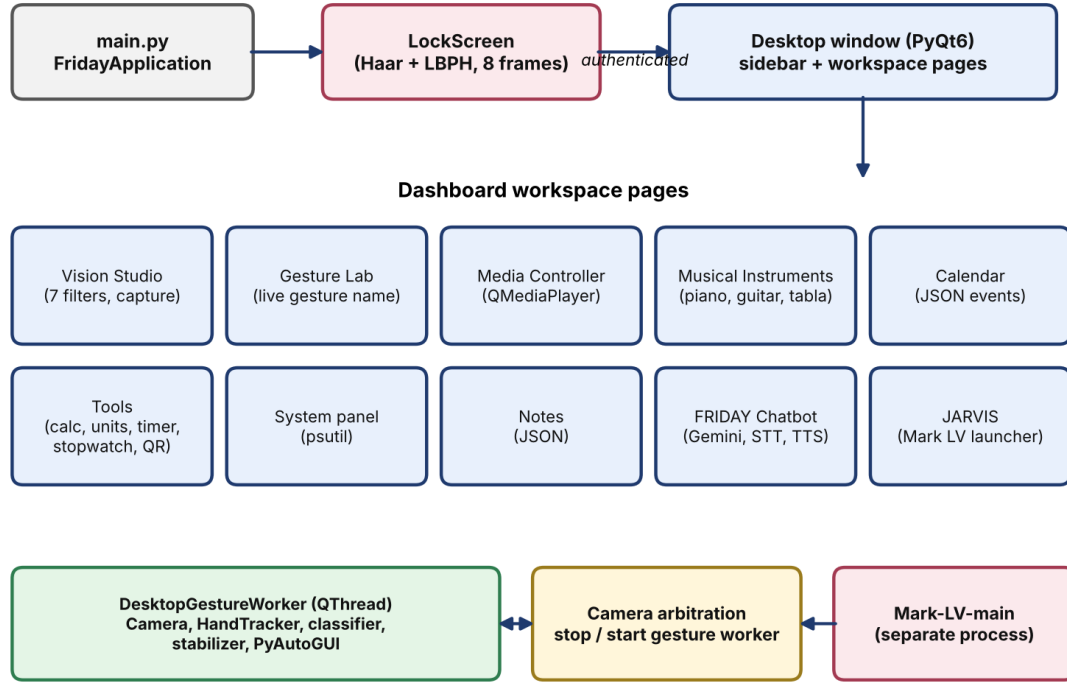


Figure 3: Module architecture of FRIDAY

5.4 Screenshot gesture and notification

When both detected hands are classified as THUMBS_UP for six consecutive frames, the worker emits TWO_THUMBS_UP once. The Desktop then calls PyAutoGUI to capture the screen, saves it as `data/screenshots/FRIDAY_<YYYYMMDD_HHMMSS>.png`, updates the hand-tracking card status, and shows a NotificationToast: a frameless, always-on-top, non-focus-stealing card that fades in for about three seconds and also raises a system notification (Figure 13).

6. Techniques / Algorithms Used

6.1 Image processing

Technique	Use in FRIDAY (parameters from the implementation)
Grayscale conversion	BGR to gray before face detection, recognition and thresholding; reduces data to one intensity channel.
Histogram equalisation	<code>cv2.equalizeHist</code> on the 200×200 face crop to improve contrast and reduce illumination dependence.
Filtering (smoothing)	Gaussian blur with a 15×15 kernel (Vision Studio).
Edge detection	Canny with hysteresis thresholds 100 and 200 on the grayscale frame.
Thresholding	Global binary threshold at 127 (output 0 or 255): $s = 255$ if $r \geq 127$, else 0.
Sharpening	<code>filter2D</code> with a kernel built from a 5×5 Gaussian ($\sigma = 1$): the negative Gaussian with 2 added at the centre, i.e. $I + (I - \text{blurred})$ in a single convolution.
Colour-space conversion	BGR to HSV (Vision Studio); BGR to RGB for MediaPipe and for Qt display.
Negative transformation	Intensity inversion $s = 255 - r$ using <code>cv2.bitwise_not</code> .

Technique	Use in FRIDAY (parameters from the implementation)
Geometric / drawing	Horizontal mirroring (cv2.flip); landmark skeleton, key, string and drum-zone overlays drawn with cv2 line, circle, ellipse and rectangle primitives.
Feature extraction	LBPH texture histograms for faces; 21 hand landmarks, finger states and a normalised thumb–index pinch ratio for hands.

6.2 Detection, recognition and classification

- **Face detection:** Haar cascade (haarcascade_frontalface_default) with scaleFactor 1.1, minNeighbors 5 and minimum face size 80×80 pixels.
- **Face recognition:** Local Binary Pattern Histograms (LBPH). The recogniser returns a distance (lower is better); a match requires distance below 60, a starting value that can be calibrated.
- **Finger states:** index, middle, ring and little fingers are "up" when the tip is above the PIP joint (smaller y); the thumb is "up" when its tip is more than 1.15 times as far from the wrist as the thumb IP joint.
- **Gesture rules (in order):** PINCH if the 3-D thumb-tip to index-tip distance divided by the palm width (landmark 5 to landmark 17) is below 0.42, which makes the test independent of how far the hand is from the camera; then OPEN_PALM (all five up), FIST (none up), POINT (index only), TWO_FINGER (index and middle), THUMBS_UP (thumb only), THREE_FINGER and FOUR_FINGER (by finger count); otherwise UNKNOWN.
- **Two-hand gesture:** TWO_THUMBS_UP is produced when two hands are visible and both are classified THUMBS_UP for six consecutive frames.

6.3 Video processing

- **Frame extraction:** continuous frame acquisition from the webcam with mirroring (cv2.flip).
- **Tracking:** MediaPipe Hand Landmarker in VIDEO mode with up to two hands and detection, presence and tracking confidences of 0.5.
- **Temporal filtering:** authentication requires 8 consecutive matching frames; the desktop gesture worker requires 4 identical consecutive classifications (the stabilizer default is 6); the cursor position is smoothed by exponential smoothing with factor 0.28 and a 3-pixel dead-zone.
- **Pointer mapping:** the index fingertip is mapped from the central 14–86% region of the camera frame to the full screen, so the screen edges can be reached without leaving the camera view; the region is drawn as "POINTER ACTIVE AREA" on the preview.
- **Live processing:** all Vision Studio operations are applied frame by frame with an on-screen FPS counter.

6.4 Audio synthesis for the instruments

The instruments do not use recorded samples. At start-up, Pygame's mixer is initialised at 44.1 kHz, 16-bit, stereo and the sounds are pre-synthesised with NumPy: piano notes as a fundamental

plus two harmonics with an exponential decay envelope; guitar strings as a fundamental plus three harmonics with a slower decay; and tabla strokes (DHA, DHI, NA, TIN) as decaying sine tones with a small noise component. Pre-computing the buffers keeps the trigger path short when a fingertip enters a key, string or drum zone.

7. Implementation

Item	Details
Language	Python 3.12
Image / video	OpenCV (opencv-contrib-python 4.11.0.86, which provides LBPH), NumPy 1.26.4, mss
Hand tracking	MediaPipe 0.10.21 (Hand Landmarker task model)
Interface	PyQt6 (QThread workers, QTimer, signals and slots, QtMultimedia)
Desktop control	PyAutoGUI (cursor movement, click, screenshot, media keys)
Audio	pygame (instrument synthesis and playback), pyttsx3 (speech), SpeechRecognition with PyAudio (voice input)
AI and utilities	google-genai (Gemini chat), python-dotenv (API key), psutil (system panel), qrcode with Pillow (QR generator)
Hardware (original tests)	Lenovo ThinkBook 14, AMD Ryzen 5 7535HS, 16 GB RAM, integrated AMD Radeon graphics, Windows 11, built-in webcam
Additional platform	macOS (the screenshots in this report were captured on a Mac running FRIDAY and Mark LIV)

7.1 Project structure

- **authentication/**: face_detector, face_recognizer, register_face, train_faces, login, authentication_state.
- **vision/**: camera, hand_tracker, finger_detector, gesture_classifier, gesture_stabilizer, gesture_actions, desktop_gesture_worker, and unit tests for the stabilizer and hand tracking.
- **dashboard/**: lock_screen, desktop, theme, vision_studio, gesture_lab, musical_instruments, media_controller, calendar, notes, tools, system_panel, chatbot, notification_toast, jarvis.
- **instruments/**: standalone Air Piano (airpiano.py), Air Guitar (Guitar.py) and Air Tabla (tabla.py) scripts, plus model assets.
- **models/**: hand_landmarker.task.
- **Mark-LIV-main/**: the separate Mark LIV assistant.
- **main.py**: application entry point.

7.2 Important implementation steps

1. Register the user: 20 face crops are saved only when exactly one face is visible.
2. Train the LBPH model on the preprocessed crops (sorted folder order keeps label IDs consistent between training and loading) and save it to a model file.

3. Lock screen: detect, recognise and verify over 8 frames, then emit an authenticated signal that opens the dashboard.
4. Gesture worker thread: capture, detect landmarks for up to two hands, classify, stabilize and act, emitting the preview frame and confirmed gesture to the interface.
5. Air mouse: map the index fingertip, using the 14–86% active region of the camera frame so screen edges are reachable, to screen coordinates; smooth the motion; click on pinch with a flag so one pinch gives one click.
6. Camera ownership: the Desktop stops the gesture worker while Vision Studio or the instruments use the camera and restarts it afterwards.
7. Screenshot gesture: two thumbs up for six frames triggers a PyAutoGUI screenshot saved under data/screenshots/ and a notification toast.

7.3 Gesture-to-action mapping

Gesture	Rule	Action in FRIDAY
POINT	Index finger up only	Move the pointer (acts on the raw label for responsiveness)
PINCH	Thumb–index distance / palm width < 0.42	Click / select (one click per pinch)
TWO_FINGER	Index and middle up	Toggle the FRIDAY chat panel
OPEN_PALM	All five fingers up	Click at the pointer
FIST	No fingers up	Close the active workspace and return to Home (idle)
THUMBS_UP	Thumb only	Shown in the Gesture Lab
THREE_FINGER / FOUR_FINGER	Three / four fingers up	Recognised and displayed in the Gesture Lab (no action bound)
TWO_THUMBS_UP	Both hands thumbs up, 6 frames	Save screenshot and show notification toast

7.4 Login and access control

FRIDAY starts on the LockScreen, a full-screen window that opens the camera, draws a green box and the label FACE around each detected face, and reports its state: SYSTEM LOCKED when no face is present, a multiple-face warning when more than one face is visible, "Verifying <name> • n/8" while a registered user is being matched, ACCESS DENIED with the distance when the face is unknown or the distance is not below 60, and ACCESS GRANTED with a welcome message after eight consecutive matches. Only then is the dashboard created, so no dashboard feature is reachable without a registered face. Figure 9 shows the two outcomes captured from the running system.

7.5 Dashboard modules

The dashboard uses the "Cloud Garden" theme: a sage-green sidebar with Home, Vision Studio, Gesture Lab, Media, Musical Instruments, Calendar, Tools, System, Notes and JARVIS buttons, a greeting, and a status card showing ONLINE · READY, CPU, RAM and the time. The Home page (Figure 4) shows an animated companion orb whose eyes follow the pointer, the "Today, in Bloom" card, and a Hand Tracking card with the live camera preview and gesture-control status.

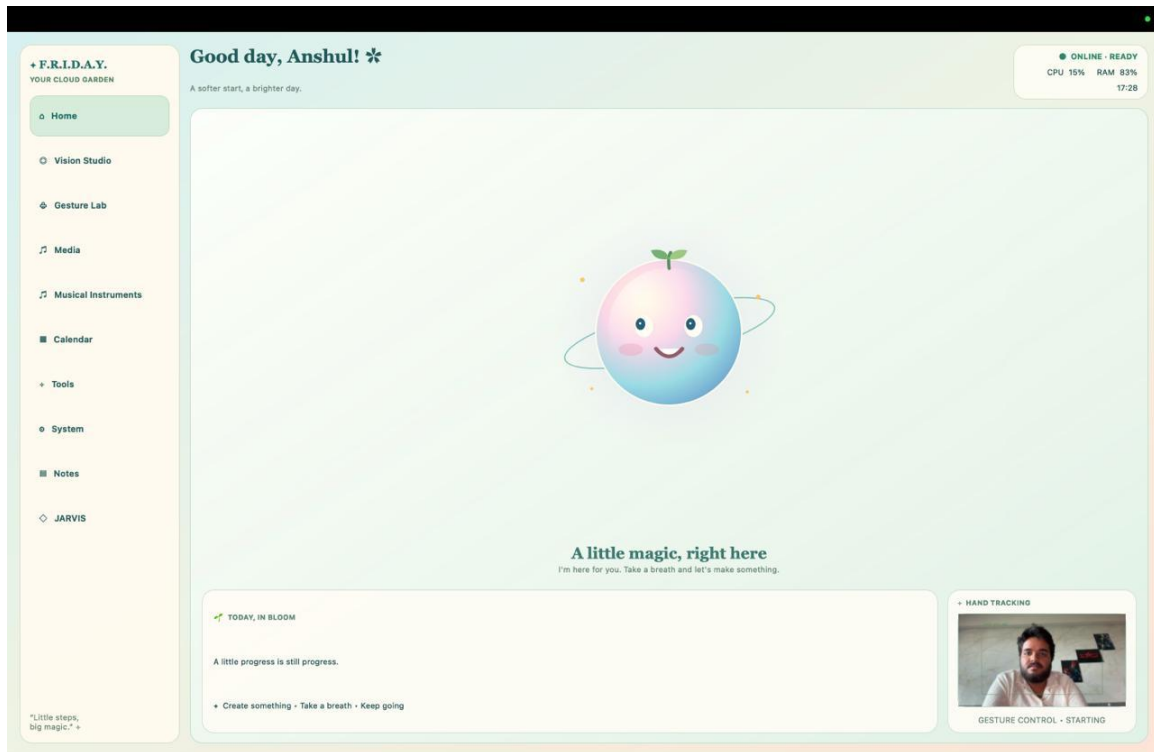


Figure 4: FRIDAY Home page with the companion orb, system status and Hand Tracking preview

Module	What it provides
Vision Studio	Live camera view with Original, Grayscale, Blur, Edge, Threshold, Sharpen, HSV and Negative modes, Capture button, mode label, FPS and resolution readout (Figure 11).
Gesture Lab	Large live display of the confirmed gesture name and a Back to Idle button (Figure 12).
Media Controller	Local playlist (mp3, wav, flac, m4a, aac, ogg, wma) played through QMediaPlayer, play queue, seek bar, previous / pause / next, volume slider, mute and system media-key control (Figure 5).
Musical Instruments	Air Piano, Air Guitar and Air Tabla played with the index fingertip on the live camera view (Section 7.6).
Calendar	Month view with today highlighted, weekends in red, per-day event list, event title, time and notes fields, Add / Delete event, stored in data/calendar_events.json (Figure 6).
Tools	Calculator, unit converter, countdown timer, stopwatch and QR-code generator.
System	Live CPU, memory, disk, battery, core-count and uptime readings from psutil.
Notes	Create, edit, save and delete notes stored in data/notes.json.

Module	What it provides
FRIDAY chat	Text and voice questions answered by Gemini with optional spoken narration (Section 7.7, Figure 7).
JARVIS	Launches the Mark LIV assistant (Section 7.8, Figure 8).

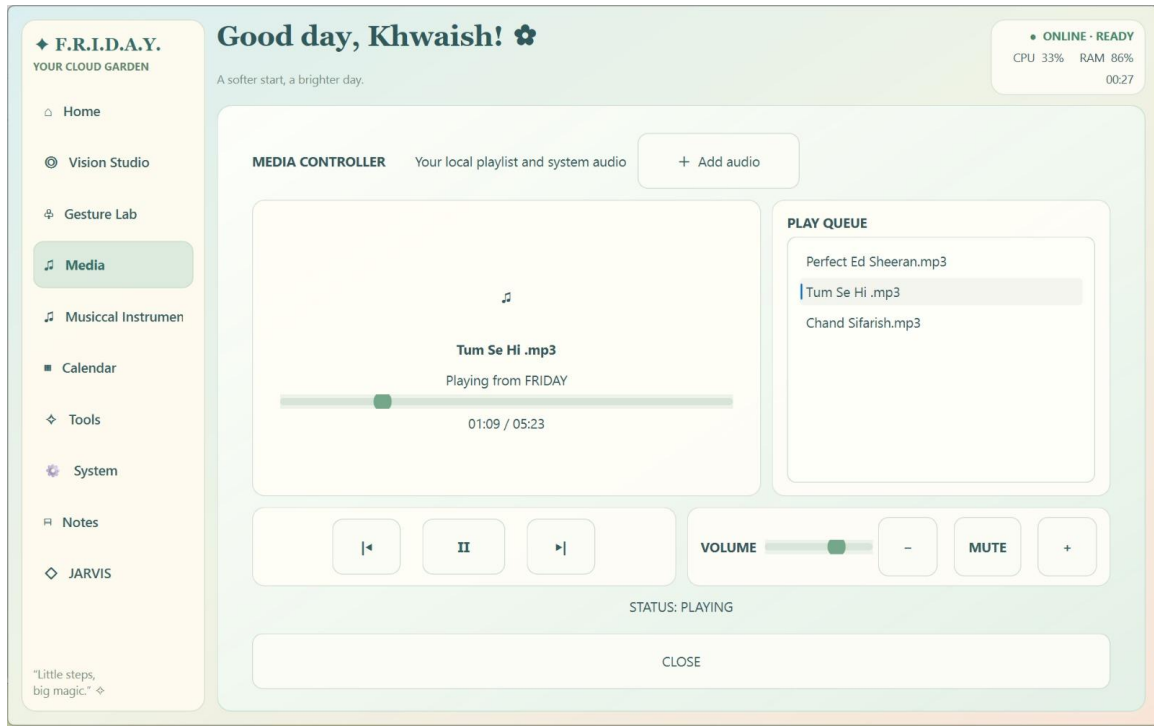


Figure 5: Media Controller playing a local track, with play queue, volume and status line

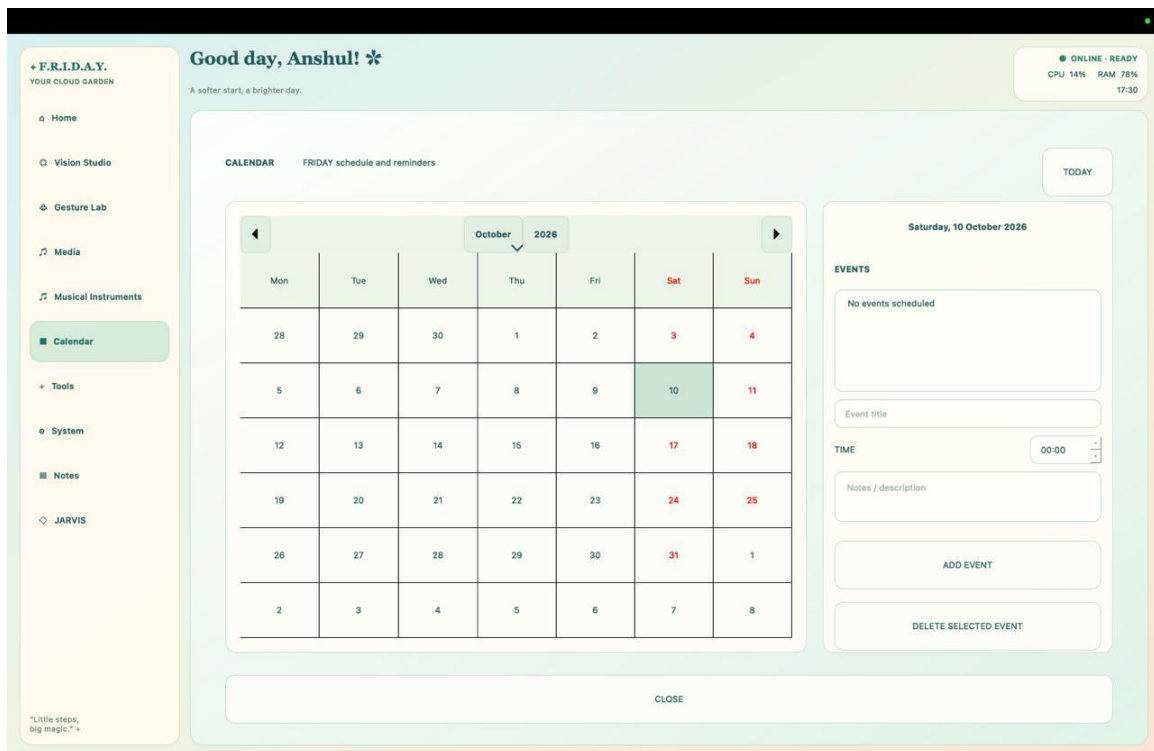


Figure 6: Calendar with the selected date and event entry panel

7.6 Virtual musical instruments

- **Air Piano:** seven keys (C, D, E, F, G, A, B) occupy the lower 38% of the frame; the key under the index fingertip is highlighted and its note plays once when the fingertip enters it.
- **Air Guitar:** six strings are drawn across the frame; touching a string plays its tone with a vibration animation.
- **Air Tabla:** a circular drum with a central syahi (DHA), and top (TIN), left (DHI) and right (NA) zones, selected by the fingertip's distance and direction from the drum centre.

All three share one camera view, one hand-landmark detector and the hand-skeleton overlay; the active instrument is chosen with buttons, and the camera is released when the page is hidden.

7.7 FRIDAY chat assistant

The chat panel is opened from the sidebar or by the two-finger gesture. Questions are sent to Google Gemini by a worker thread so the interface stays responsive. The worker walks down a ladder of Gemini models; if one is overloaded (503), rate-limited (429) or missing (404) it is placed on a short cooldown and the next model is tried, so a single model outage does not stop the assistant. The API key is read from a .env file. Voice input uses SpeechRecognition, replies can be narrated with pyttsx3 after removing markdown symbols, and the user can stop generation, listening or speaking at any time. Figure 7 shows an example exchange.

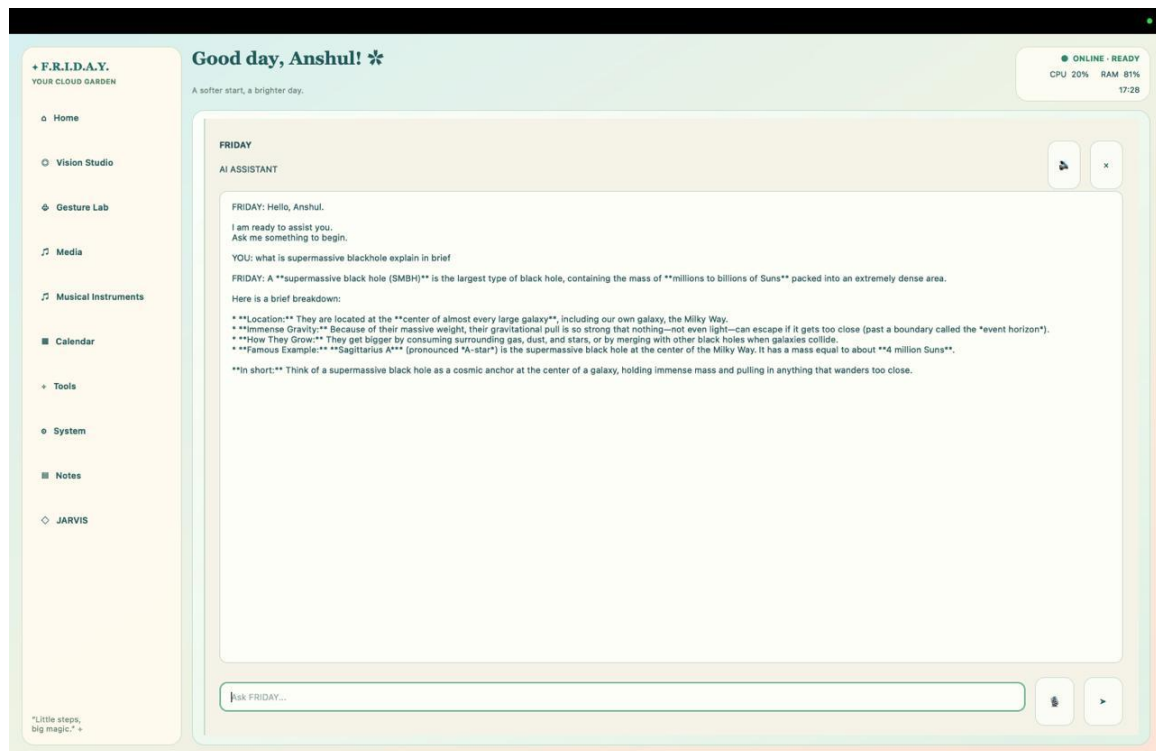


Figure 7: FRIDAY chat answering "what is supermassive blackhole explain in brief"

7.8 Mark LIV (JARVIS) integration

The JARVIS button calls `launch_mark_liv()`. FRIDAY stops its gesture worker to release the camera, starts Mark-LIV-main/main.py in a separate, detached process using the project's virtual-environment Python, and then closes itself so Mark LIV has exclusive use of the display, camera and microphone. Mark LIV is an open-source, voice-first assistant by FatihMakes built on the Gemini Live API. Its HUD (Figure 8) shows a system monitor, a central avatar with listening status, an activity log, a news panel, file upload, a command box, an Interrupt button and a microphone indicator. According to its documentation it also provides wake-word activation ("Hey Jarvis"), push-to-talk, persistent memory, computer-control skills, screen and webcam vision, and a plugin system.

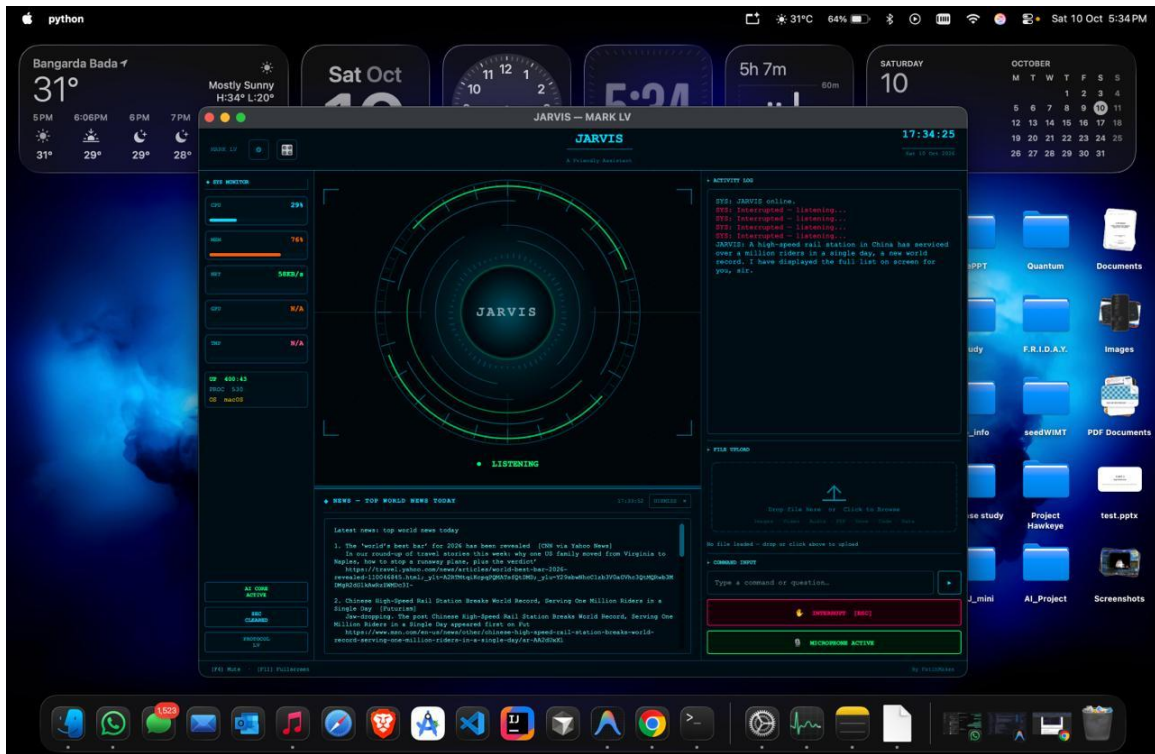


Figure 8: Mark LIV (JARVIS) HUD launched from FRIDAY (captured on macOS)

8. Results and Discussion

The figures in this section are screenshots captured from the running application. Entries marked "Not tested", "Not measured", "Not captured" or "Not shown" were not recorded during the demonstration and are left as such deliberately, so that no unmeasured value appears in the report.

8.1 Face authentication

Figure 9 shows the lock screen for a registered user and for an unregistered person. The registered user was granted access and greeted by name; the unregistered person's face was detected (green box) but rejected as "Unknown face" with a distance of 81.01, which is above the threshold of 60.

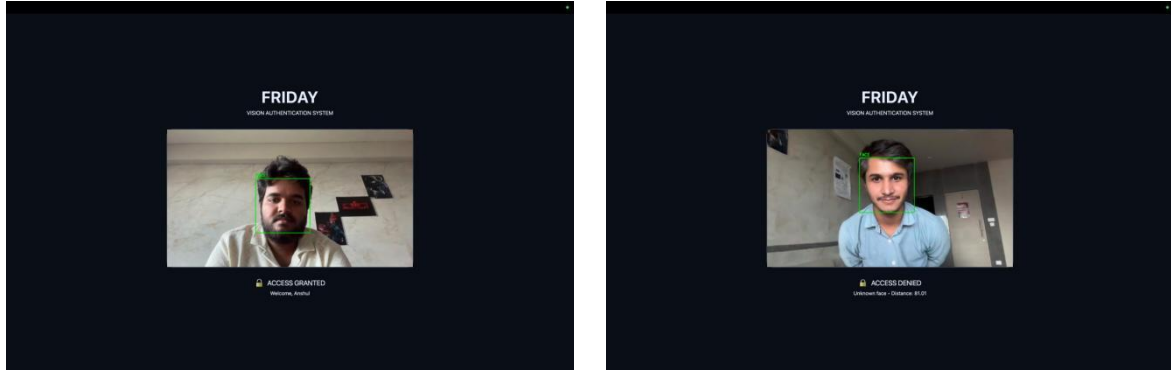


Figure 9: Lock screen – (a) ACCESS GRANTED, Welcome Anshul; (b) ACCESS DENIED, unknown face, distance 81.01

Test case	Expected	Observed	Distance
Registered user	Access granted after 8 matches	ACCESS GRANTED (Figure 9a)	Not shown
Unregistered person	Access denied	ACCESS DENIED, "Unknown face" (Figure 9b)	81.01
No face in view	Remains locked	Not tested	—
Two faces in view	Authentication rejected	Not tested	—
Dim lighting	Granted or locked (note which)	Not tested	—

8.2 Mini Vision Studio

Figure 10 is a capture saved by the Vision Studio in EDGE mode (Canny, thresholds 100 and 200) at 1280×720 . The hair texture, shoulders and the edges of the objects in the background are clearly outlined, while the smooth skin region of the face produces few edges, as expected, since Canny responds to strong intensity gradients.

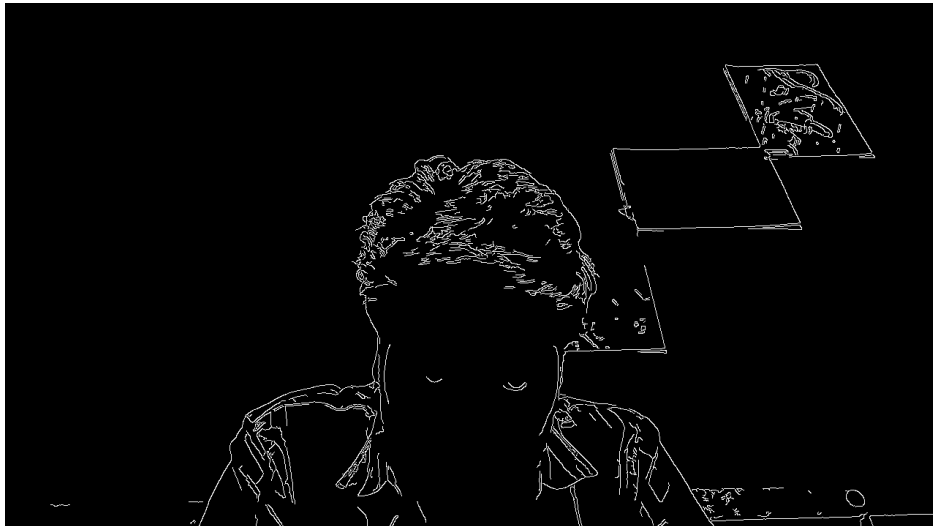


Figure 10: Edge map captured from the Mini Vision Studio (Canny, 100/200)

Figure 11 shows the live thresholding output: every pixel is mapped to black or white about the intensity 127, so the face, hair and beard fall mostly into the dark class while bright wall regions become white; the status bar reports 30.0 FPS at 1280×720 .

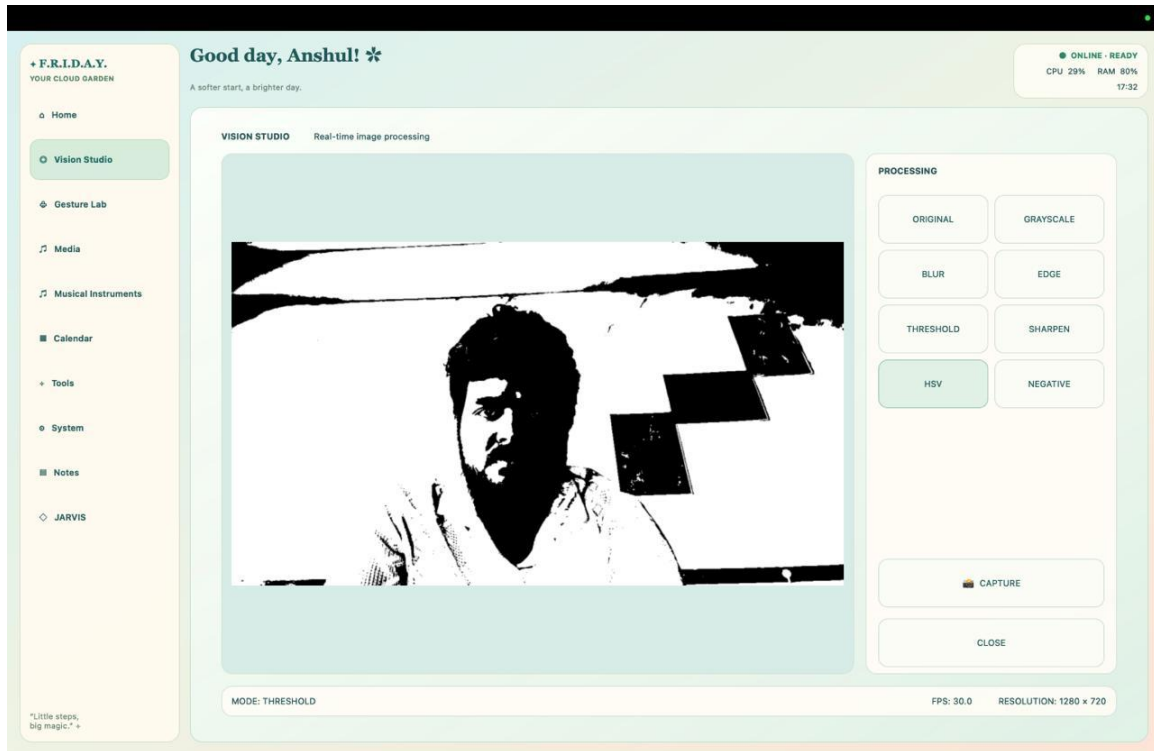


Figure 11: Vision Studio live threshold output (mode indicator *THRESHOLD*, 30.0 FPS, 1280 × 720)

Operation	Expected effect	Observation
Grayscale	Single intensity channel	Not captured
Blur (15 × 15)	Noise and fine detail suppressed	Not captured
Edge (Canny)	Thin binary edge map (Figure 10)	Edges of hair, clothing and objects clearly visible
Threshold (127)	Black/white segmentation (Figure 11)	Binary output; face and hair dark, bright wall white
Sharpen	Edges and detail enhanced	Not captured
HSV	Hue, saturation, value channels	Not captured
Negative	Inverted intensities	Not captured

8.3 Gesture recognition and screenshot capture

The Gesture Lab displays the confirmed gesture in large type; Figure 12 shows THUMBS UP being recognised. Figure 13 shows the result of the two-thumbs-up gesture: the toast "Screenshot Captured – Saved: FRIDAY_20261010_174315.png" appears at the top-right of the screen and the Hand Tracking card reports "GESTURE CONTROL • SCREENSHOT SAVED ✓".

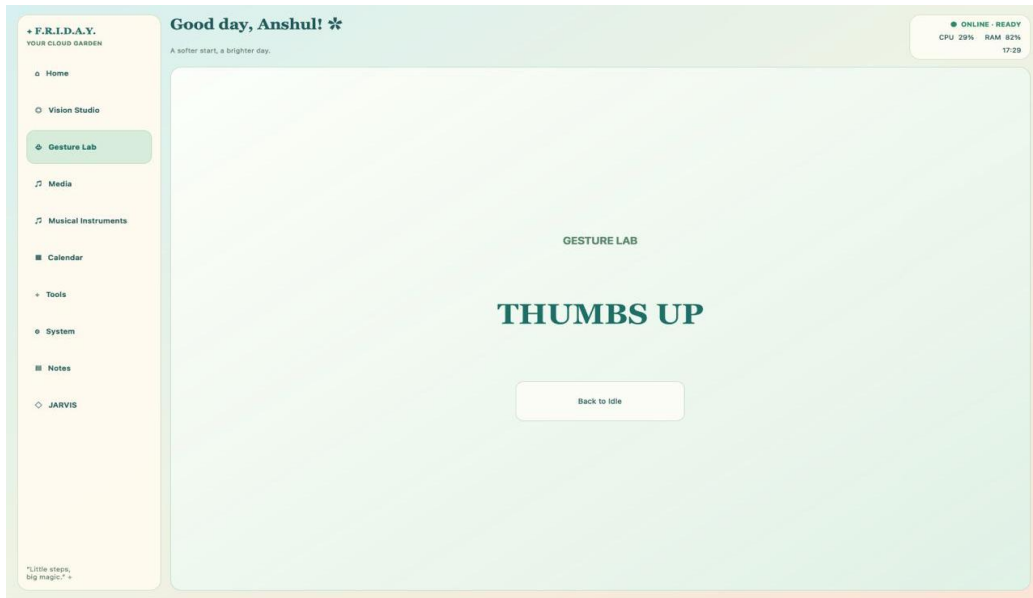


Figure 12: Gesture Lab showing the recognised gesture THUMBS UP

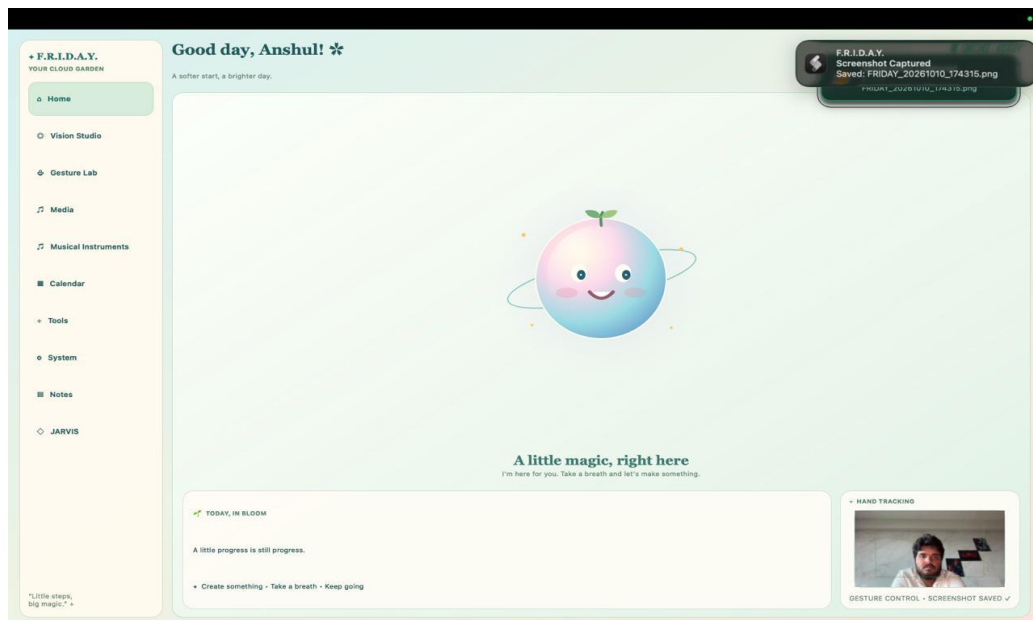


Figure 13: Screenshot gesture – notification toast and Hand Tracking status

Gesture	Rule	Action in FRIDAY	Detected? (Y/N)
POINT	Index up only	Cursor movement	Not tested
PINCH	Thumb–index / palm width < 0.42	Mouse click	Not tested
TWO_FINGER	Index + middle up	Toggle chat	Not tested
OPEN_PALM	All five fingers up	Click	Not tested
FIST	No fingers up	Return to Home	Not tested
THUMBS_UP	Thumb up only	Gesture Lab label	Y (Figure 12)
TWO_THUMBS_UP	Both hands thumbs up, 6 frames	Screenshot + toast	Y (Figure 13)

This table records observed behaviour. Classification accuracy should only be quoted if a controlled test is run (for example 20 trials per gesture), in which case a confusion matrix can be added here.

8.4 Dashboard modules in use

Figures 5 to 8 show the dashboard modules working: the Media Controller playing a track at 00:41 of 03:21 with the status line "VOLUME UP"; the Calendar on Saturday 10 October 2026 with no events; the chat assistant returning a structured answer; and the Mark LIV HUD with its activity log, news panel and listening indicator. In the chat screenshot the Gemini reply shows markdown asterisks as literal text because the panel displays plain text.

8.5 Quantitative performance

Metric	Value	Method
Vision Studio frame rate	30.0 FPS at 1280×720	Status bar of Figure 11
System CPU usage while FRIDAY ran	14% – 29%	Dashboard status card across the captured screens (system-wide, not a controlled test)
System RAM usage while FRIDAY ran	78% – 83%	Dashboard status card across the captured screens (system-wide)
Face detection FPS	Not measured	Frames / s over 60 s
Hand tracking + classification FPS	Not measured	Frames / s over 60 s
End-to-end gesture latency (ms)	Not measured	Timer around loop
Authentication time (s)	Not measured	Face visible to granted

Because the gesture stabilizer needs 4 identical frames, a new gesture needs at least 4 processed frames before it is confirmed (6 for two thumbs up); the frame rate therefore directly sets the minimum response time. The pointer itself acts on the raw POINT label, so cursor motion is not delayed by the stabilizer.

9. Challenges / Limitations

- **Lighting:** Haar detection and LBPH are sensitive to low light, backlighting and shadows; histogram equalisation helps but does not remove the problem.
- **Recognition threshold:** LBPH returns a distance, not a probability. The value 60 is an initial setting and must be calibrated with genuine and impostor trials.
- **Spoofing and scale:** There is no liveness detection, so a photograph could be a risk, and LBPH is suited only to a few registered users.
- **Gesture ambiguity:** Finger states use simple geometry. A tilted or rotated hand can mislabel fingers, and a closed pinch can resemble other poses; the thumb rule is the least reliable.
- **Flicker:** Per-frame labels fluctuate; the 4-frame stabilizer solved this at the cost of a small delay.

- **Latency and jitter:** Landmark detection, mapping and GUI updates add delay to the air mouse; exponential smoothing and a dead-zone reduce jitter but add lag.
- **Camera sharing:** The gesture worker, Vision Studio and the instruments all need the single webcam, so ownership is handed over by stopping and restarting the worker, with a brief pause in gestures.
- **Network and API dependence:** The chat assistant needs an internet connection and a Gemini API key, and the model ladder can still fail when every model is overloaded. Voice features need a working microphone and speakers.
- **Platform differences:** Cursor control, screenshots, media keys and the speech and audio libraries depend on operating-system permissions and drivers; the documented setup targets Windows, while the screenshots here were taken on macOS.
- **Chat formatting and capture location:** The chat panel shows markdown asterisks literally, and Vision Studio captures are written to the working directory rather than a dedicated folder.
- **Dataset limits:** Only a small set of locally captured faces from one environment is used, so results may not generalise to other conditions.

10. Conclusion

FRIDAY integrates image processing, video processing and computer vision into one real-time desktop application. Face authentication was implemented with Haar-cascade detection, preprocessing, LBPH recognition and 8-frame verification and gates access to the dashboard. Gesture control was implemented with MediaPipe hand landmarks, finger-state analysis, a scale-normalised pinch test, a rule-based classifier and a 4-frame temporal stabilizer, driving an air mouse, the chat panel and a two-hand screenshot gesture with an on-screen notification. The Mini Vision Studio demonstrates grayscale conversion, Gaussian smoothing, Canny edge detection, thresholding, sharpening, HSV conversion and negative transformation on live video, and the Air Piano, Guitar and Tabla extend the same landmark stream into a musical interface. Around these sit a Gemini chat assistant with voice, media, calendar, notes, utility and system tools, and a hand-over to the Mark LIV (JARVIS) assistant.

The project gave practical experience with camera acquisition, preprocessing, feature extraction, classification, temporal filtering, threading, audio synthesis and real-time performance trade-offs. Future work includes liveness detection, a deep-learning face embedding, learned or dynamic gestures, custom gesture bindings, multi-hand chords for the instruments, an offline language model, and a systematic accuracy evaluation.

Appendix A: Installation and Running

Source code: <https://github.com/Auto-KAD/F.R.I.D.A.Y>. Python 3.12 is recommended; a webcam is required for sign-in and gestures, a microphone and speakers for voice features, and an internet connection with a Google Gemini API key for the chat assistant.

1. **Create the environment and install dependencies:** `python -m venv .venv`, activate it, then `python -m pip install -r requirements.txt`.
2. **Enroll a face:** run `python -m authentication.register_face`, then `python -m authentication.train_faces`. Images and the trained model are stored under `data/`.
3. **Configure the chat assistant:** create a `.env` file in the repository root containing `GEMINI_API_KEY=<your key>`. The file is excluded from Git.
4. **Run FRIDAY:** `python main.py`. The lock screen opens first and the dashboard appears after the enrolled user is recognised.
5. **Optional – Mark LIV:** in `Mark-LIV-main` run `python setup.py` once; the JARVIS sidebar button then launches it, and it asks for its own Gemini key on first start.

Local data and privacy

Face images, the trained face model, screenshots, notes and calendar data are stored under `data/`, which is ignored by Git. Camera frames are processed locally for face and gesture features; chat prompts are sent to Google Gemini only when the chat assistant is used. API keys and personal face data must not be committed.

References

- [1] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.
- [2] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in Proc. IEEE CVPR, 2001, pp. I-511–I-518.
- [3] T. Ahonen, A. Hadid and M. Pietikäinen, "Face description with local binary patterns: Application to face recognition," IEEE Trans. Pattern Anal. Mach. Intell., vol. 28, no. 12, pp. 2037–2041, 2006.
- [4] F. Zhang et al., "MediaPipe Hands: On-device real-time hand tracking," arXiv:2006.10214, 2020.
- [5] J. Canny, "A computational approach to edge detection," IEEE Trans. Pattern Anal. Mach. Intell., vol. PAMI-8, no. 6, pp. 679–698, 1986.
- [6] R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th ed. Pearson, 2018.
- [7] OpenCV documentation: image processing, video I/O and face module (LBPHFaceRecognizer), docs.opencv.org.
- [8] Google MediaPipe documentation: Hand Landmarker, ai.google.dev/edge/mediapipe.
- [9] Riverbank Computing, PyQt6 documentation; PyAutoGUI documentation; pygame documentation.
- [10] Google, Gemini API documentation (google-genai SDK), ai.google.dev.
- [11] FatihMakes, Mark LIV (JARVIS) open-source assistant, bundled in Mark-LIV-main of the FRIDAY repository.
- [12] Auto-KAD, F.R.I.D.A.Y. source repository, <https://github.com/Auto-KAD/F.R.I.D.A.Y>.